

QA AUTOMATION PORTFOLIO

MarcomBox DAM End-to-End Test Automation

A complete Playwright + TypeScript automation framework for the MarcomBox Digital Asset Management (DAM) platform — covering asset upload, metadata management, search, sharing, guest upload, and CI/CD delivery.

PROJECT

Digital Asset Management QA Automation

APPLICATION

MarcomBox DAM (QA Environment)

ROLE

QA Automation Engineer

GENERATED

August 6, 2026

1. Project Overview

This project automates critical user journeys in a Digital Asset Management (DAM) system used for storing, organizing, and sharing marketing assets. The goal was to build a reliable, maintainable, and scalable test automation suite that validates real business workflows — not just UI clicks.

Problem

Manual regression of DAM workflows (upload, metadata, search, share, guest upload) was slow, repetitive, and error-prone across releases.

Solution

Built a Playwright automation framework with Page Object Model, parallel execution, email verification, Docker packaging, and GitHub Actions CI/CD.

2. Tech Stack

Playwright

TypeScript

Node.js

Page Object Model

Docker

Docker Compose

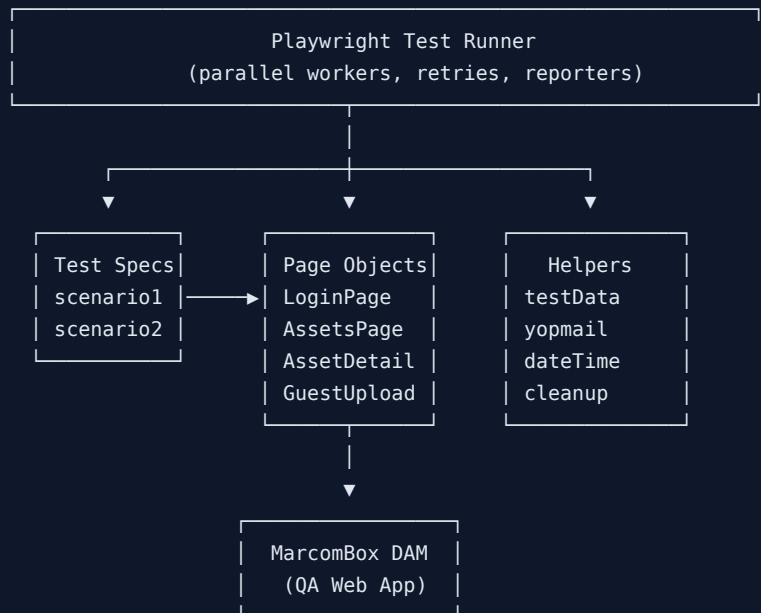
GitHub Actions

dotenv

JUnit / HTML Reports

Yopmail Email Testing

3. Architecture



Delivery: Docker → GitHub Actions → Artifact Reports

4. Project Journey — What I Built

Phase 1 — Test Framework Setup

- Initialized Playwright + TypeScript automation project
- Configured environment variables via dotenv (.env)
- Set up Playwright config with parallel workers, timeouts, and reporters
- Added HTML, JUnit, trace, screenshot, and video reporting

Phase 2 — Page Object Model (POM)

- LoginPage — multi-step authentication flow
- AssetsPage — folder navigation, search, edit mode, guest share
- AssetDetailPage — upload, metadata, edit, download, share, delete
- GuestUploadPage — external guest upload flow with OTP verification

Phase 3 — Helper Utilities

- testData.ts — unique test IDs per run (S1-VIDEO, S2-IMAGE)
- yopmail.ts — browser-based email inbox reader for share links and OTP
- dateTime.ts — metadata date/time formatting
- cleanup.ts — safe post-test asset cleanup utilities

Phase 4 — End-to-End Test Scenarios

- Scenario 1: Full MP4 asset lifecycle — upload, metadata, edit, search, download, email share, delete
- Scenario 2: Guest folder upload — invite via email, OTP login, JPG upload, DAM verification
- Both scenarios run independently in parallel with 2 workers

Phase 5 — Docker Containerization

- Dockerfile based on official Playwright v1.62.1 image
- docker-compose.yml with env injection and report volume mounts
- npm scripts: docker:build and docker:test

Phase 6 — CI/CD Pipeline

- GitHub Actions workflow on push, PR, and manual trigger
- Tests run inside Docker on ubuntu-latest runners
- GitHub Secrets for secure credential management
- Playwright report uploaded as downloadable artifact after every run
- CI retries enabled for improved stability

5. Test Scenarios Covered

Scenario 1 — Local MP4 Upload & Full Asset Lifecycle

1. Sign in and navigate to user folder in DAM > Assets
2. Upload a uniquely named MP4 file with drag-and-drop support
3. Fill metadata: title, type, date, description, tags, hyperlink
4. Save, verify, then edit title/date/description
5. Search asset, download, share via email, verify inbox
6. Delete asset and logout

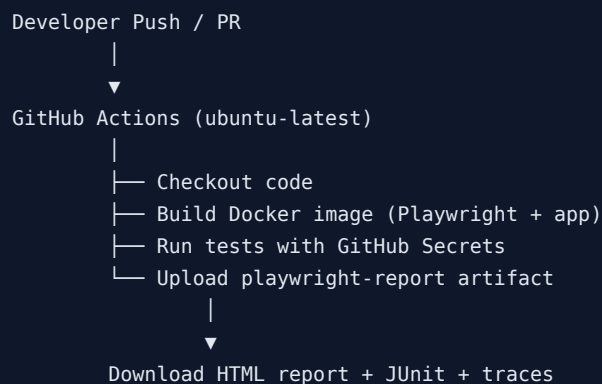
Scenario 2 — Guest Folder Upload via Email Invite

1. Sign in and enable edit mode in Assets
2. Right-click folder → Guest upload/share → send invite email
3. Open share link from Yopmail inbox
4. Enter OTP from second email
5. Upload JPG via Browse Files on guest page
6. Verify upload in DAM folder and cleanup

6. Key Engineering Decisions

- **Playwright** — chosen for auto-waiting, parallel workers, and rich debugging (trace/video)
- **Page Object Model** — keeps selectors separate from test logic for maintainability
- **Unique test data per run** — prevents duplicate upload failures
- **Yopmail browser client** — validates email flows without paid API keys
- **Independent scenarios** — separate spec files with distinct ID prefixes for safe parallel runs
- **Docker + CI/CD** — reproducible test environment and automated delivery pipeline

7. CI/CD & DevOps Workflow



8. Project Structure

```
marcombox-dam-automation/
├── src/
│   ├── helpers/          # testData, yopmail, dateTime, cleanup
│   └── pages/             # Login, Assets, AssetDetail, GuestUpload
├── tests/
│   ├── scenario1-local-upload.spec.ts
│   └── scenario2-folder-upload.spec.ts
├── test-data/            # sample.mp4, sample.jpg
├── scripts/              # PDF report generator
├── Dockerfile
├── docker-compose.yml
├── .github/workflows/ci.yml
└── playwright.config.ts
```

9. Skills Demonstrated

Playwright E2E Testing

TypeScript

Page Object Model (POM)

Test Automation Architecture

Parallel Test Execution

Email Flow Testing (Yopmail)

Docker & Docker Compose

GitHub Actions CI/CD

HTML / JUnit Reporting

Trace & Video Debugging

Environment & Secrets Management

Digital Asset Management (DAM) QA

10. Deliverables

- Production-ready Playwright + TypeScript automation framework
- 2 full end-to-end DAM workflow test scenarios
- Reusable Page Object Model and helper library
- Email-based share and OTP verification automation
- Dockerized test execution environment
- GitHub Actions CI/CD with secure secrets and downloadable reports
- HTML, JUnit, trace, screenshot, and video reporting
- Complete project documentation (README)